1. Repository Pattern

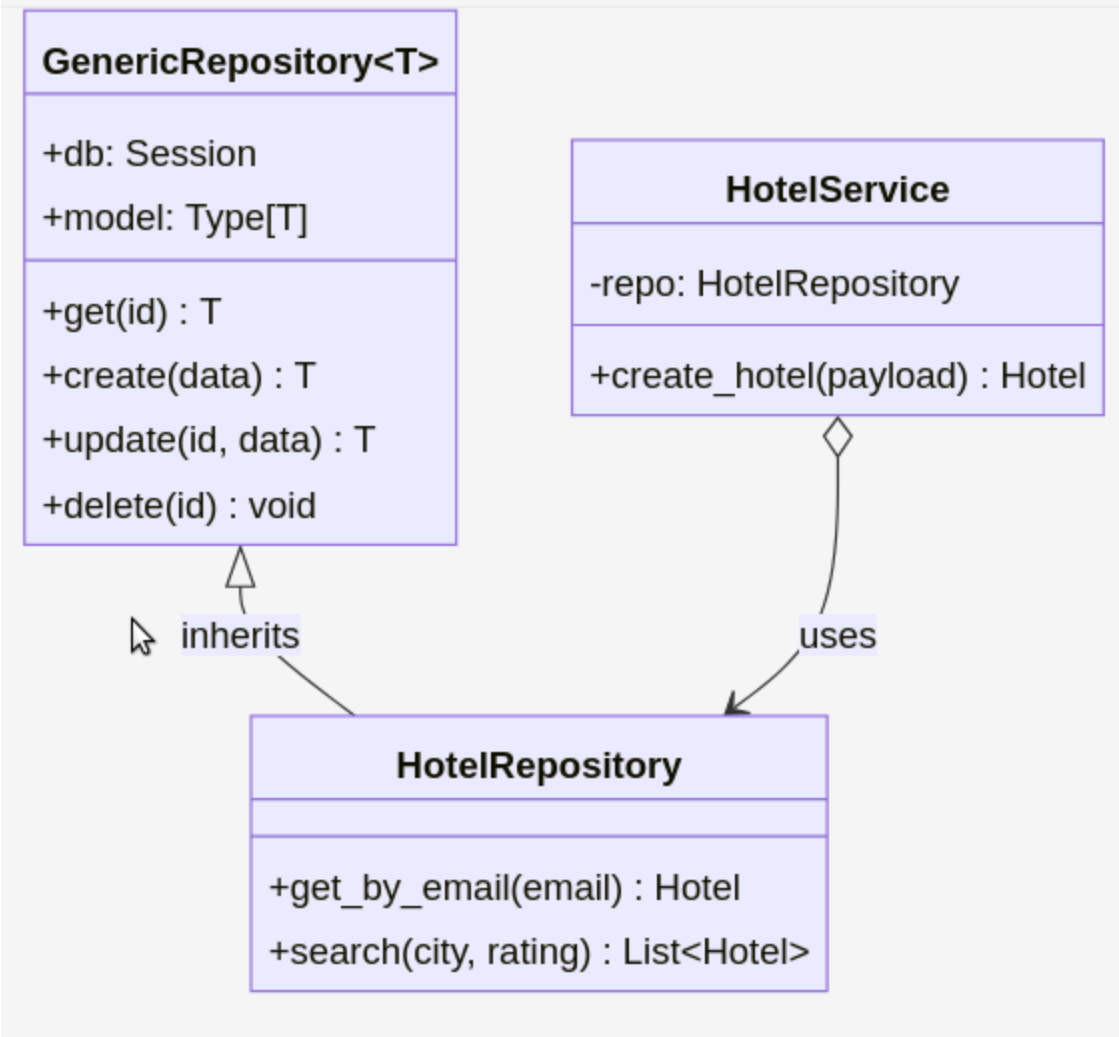
The Problem It Solves

Initially, data access logic (SQL queries) was scattered throughout the business logic. This tightly coupled the services to the specific database ORM (SQLAlchemy), making the code hard to test, hard to maintain, and violating the Single Responsibility Principle. If we ever needed to switch from PostgreSQL to MongoDB, we would have to rewrite the entire service layer.

Specific Files/Classes Involved

- **Interface/Base:** `app/repositories/base.py` (`GenericRepository[T]`)
- **Concrete Implementations:**
  - `app/repositories/hotel_repository.py` (`HotelRepository`)
  - `app/repositories/booking_repository.py` (`BookingRepository`)
  - `app/repositories/payment_repository.py` (`PaymentRepository`)
- **Consumer:** `app/services/*` (e.g., `HotelService` injects and uses `HotelRepository`)

UML Diagram & Structure



Explanation of Structure

- **`GenericRepository[T]`**: A generic base class that provides standard CRUD operations (Create, Read, Update, Delete) for any SQLAlchemy model. It abstracts the `Session` object.
- **Concrete Repositories (e.g., `HotelRepository`)**: Inherit from the base class and add entity-specific queries (e.g., `search_by_city()`).
- **Services**: The service layer (e.g., `HotelService`) receives the repository via dependency injection. It contains business rules but has no knowledge of SQL or SQLAlchemy sessions.

2. Strategy Pattern

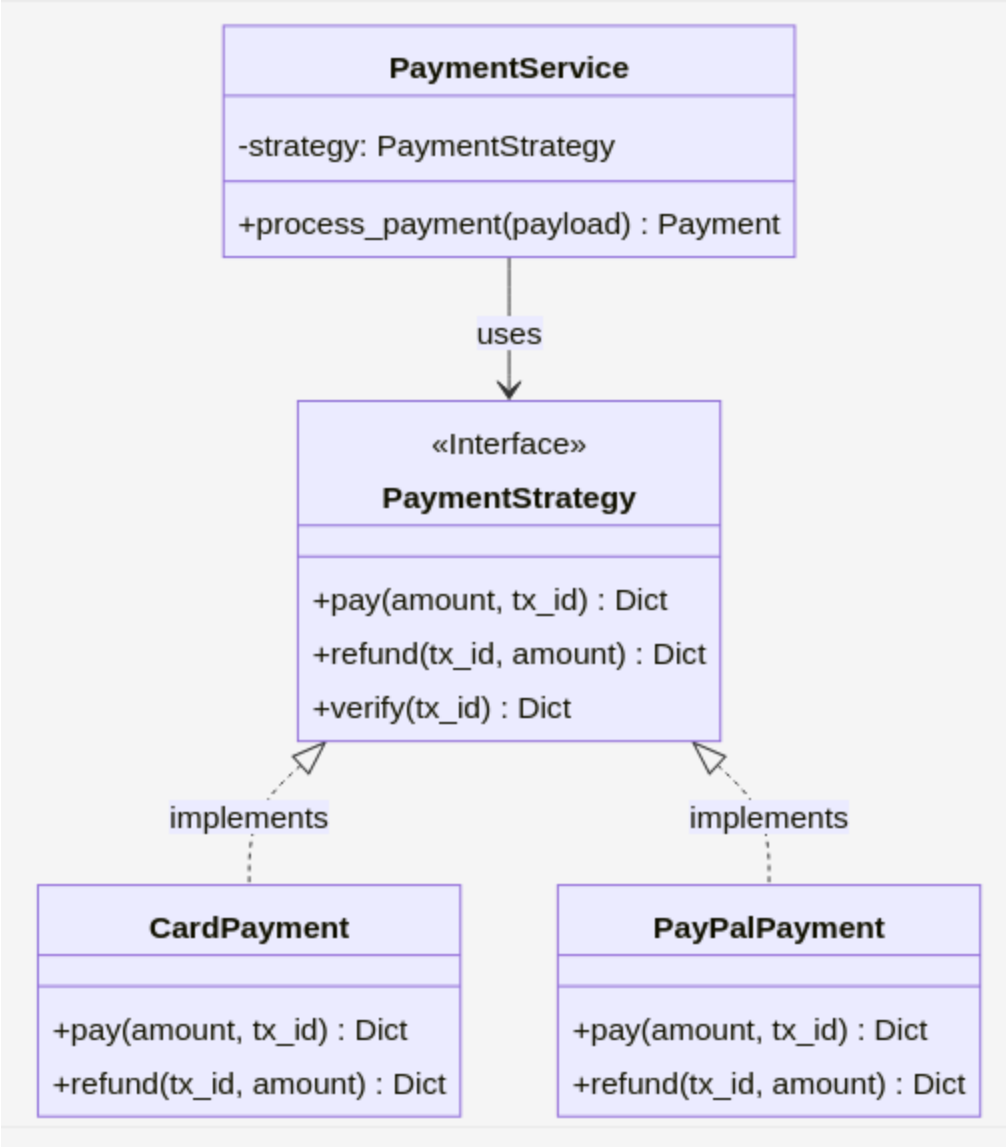
The Problem It Solves

The system needs to process payments using multiple methods (Credit Card, PayPal, Bank Transfer). Hardcoding this logic using `if/else` statements inside the `PaymentService` would make the class massive, violate the Open/Closed Principle (every new payment method requires modifying the service), and make testing individual gateways difficult.

Specific Files/Classes Involved

- **Strategy Interface:** `app/strategies/payment_strategy.py` (`PaymentStrategy`)
- **Concrete Strategies:**
  - `app/strategies/card_payment.py` (`CardPayment`)
  - `app/strategies/paypal_payment.py` (`PayPalPayment`)
  - `app/strategies/bank_transfer_payment.py` (`BankTransferPayment`)
- **Consumer:** `app/services/payment_service.py` (`PaymentService`)

UML Diagram & Structure



Explanation of Structure

- **`PaymentStrategy`**: An abstract base class (interface) that defines the contract for all payment processors (`pay`, `refund`, `verify`).
- **Concrete Strategies**: Each class implements the interface with its own specific API logic (e.g., `CardPayment` handles Stripe API simulation, `PayPalPayment` handles PayPal API simulation).
- **`PaymentService`**: Does not know *which* payment method it is using. It just calls `strategy.pay()`. This allows payment algorithms to be swapped at runtime without changing the service code.

3. Factory Pattern

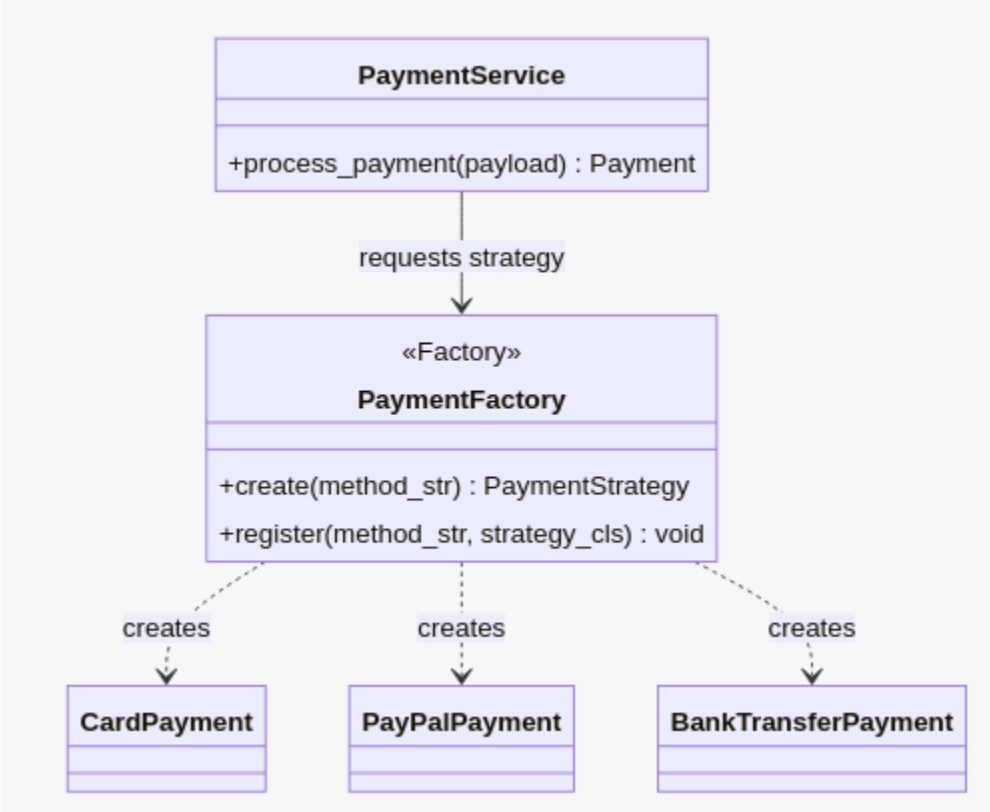
The Problem It Solves

While the Strategy Pattern allows interchangeable payment algorithms, the application still needs a way to decide *which* strategy to instantiate based on user input (e.g., if user selects "card", instantiate `CardPayment`). Putting this instantiation logic (`if method == 'card': ...`) in the service couples the service to concrete classes again.

Specific Files/Classes Involved

- **Factory Class:** `app/strategies/payment_factory.py` (`PaymentFactory`)
- **Products:** `CardPayment`, `PayPalPayment`, `BankTransferPayment`
- **Consumer:** `app/services/payment_service.py` (`PaymentService`)

UML Diagram & Structure



Explanation of Structure

- **`PaymentFactory`**: Contains a registry (dictionary) mapping string keys (e.g., `"card"`, `"paypal"`) to their corresponding Strategy classes.

- **`create(method)`**: A class method that takes the user's input string, looks up the correct class in the registry, and returns an instance of that strategy.
- **`PaymentService`**: Calls `PaymentFactory.create(payload.payment_method.value)`. It delegates the responsibility of object creation to the Factory. If a new payment method (like "Crypto") is added later, the developer only registers it in the Factory; the `PaymentService` code remains untouched.

4. Singleton Pattern

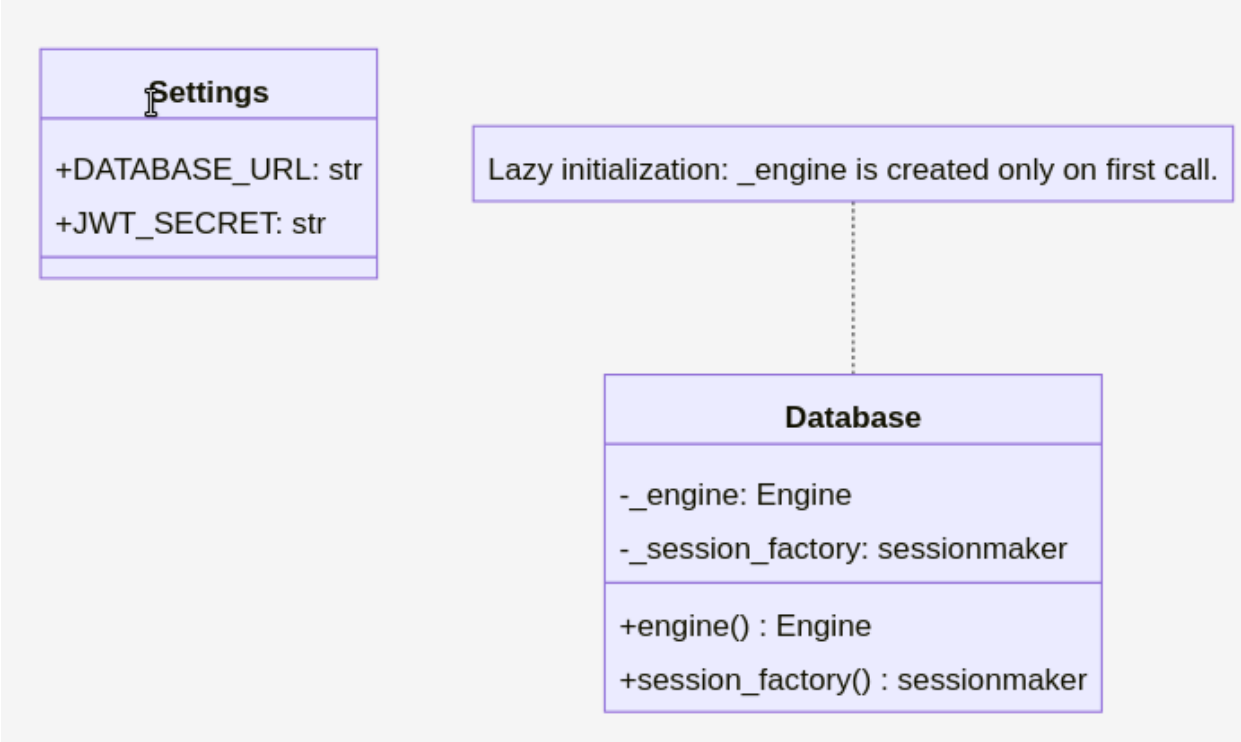
The Problem It Solves

Application configuration (database URLs, JWT secrets) and heavy resources (Database Engine, Logger) should only be created **once** during the application's lifecycle. Creating multiple database engines or parsing the `.env` file on every request wastes memory and connection pools.

Specific Files/Classes Involved

- **Config:** `app/config.py` (`Settings`, `get_settings()`)
- **Database:** `app/database.py` (`Database` class)
- **Logger:** `app/core/logger.py` (`_SingletonLogger`)

UML Diagram & Structure



Explanation of Structure

- **`Database` class:** Uses class-level variables (`_engine`, `_session_factory`) and class methods. When `Database.engine()` is called for the first time, it creates the SQLAlchemy engine. On subsequent calls, it returns the existing engine.
- **`get_settings()`:** Uses Python's `@lru_cache` decorator. FastAPI calls this function to get the `Settings` object. `lru_cache` ensures the `Settings` object is instantiated only once and cached for the lifetime of the app.